



Moving to Linked List

layvathna@gmail.com

Objective

- Understand the Doubly Linked List (DLL) ADT and its memory model.
- Implement core operations: create, traverse ($\rightarrow/\leftarrow$), insert, delete, search.
- Compare DLL vs Singly LL vs Dynamic Array (complexity & constant factors).
- Practice safe pointer patterns (sentinels, RAII, edge-case defense).
- Benchmark one or two operations to build empirical intuition.

Outcome: you can reason about when DLL is the right tool (e.g., frequent middle deletions with backward traversal needs).



head-only vs head+tail

- What is head? what is tail?
- If we maintain only head, what gets slower, what stays the same?
- If we also keep tail, which operations become $O(1)$ that weren't before?



Scenario: Tail → Head

- Story: You're building a playlist controller for a café. The list must support:
 - **Next/Prev** track from any position (including from the last track back to the first without branching logic).
 - **Continuous loop** during opening hours; no special case at the ends.
 - **Occasional removal** of the current track while playing (skip a problematic file) and continue smoothly.



Scenario: Tail $\rightarrow$ Head

- Operations needed
 - `next()` from `tail` should jump to `head` in $O(1)$.
 - `prev()` from `head` should jump to `tail` in $O(1)$.
 - Single traversal pass that can wrap without checking for `nullptr`.



Why not a plain SLL?

- SLL ends at `nullptr`
 - requires `if-else` at boundaries and can't go `prev` without searching for the predecessor.
- Design idea
 - Link the `last node's next` back to the `first node` so moving from the last song to the first is constant time ($O(1)$). So how do we stop the traversal?
 - Each node kept a forward reference to its next neighbor, moving from the first song to the second song could be constant time. So what can we do for moving backward?



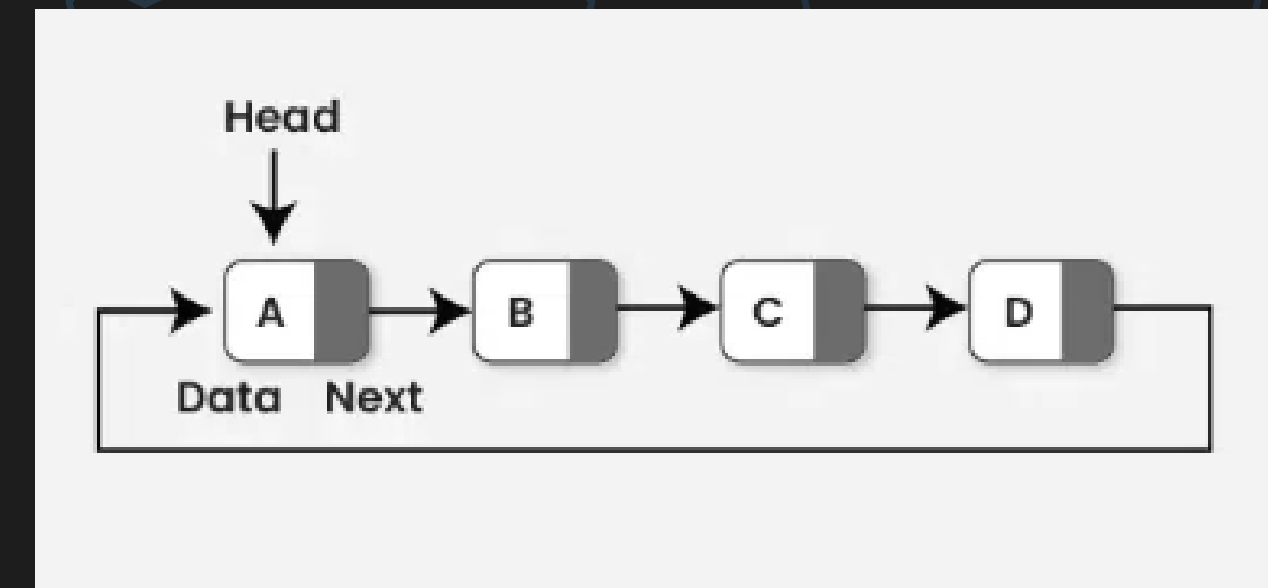
Why Circular Linked Lists?

- Some workflows are inherently cyclic: round-robin CPU scheduling, playlists, multiplayer turns...
- Avoids explicit NULL ends; simplifies wrap-around logic; constant-time move from last → first.



CSLL Concept & Node Layout

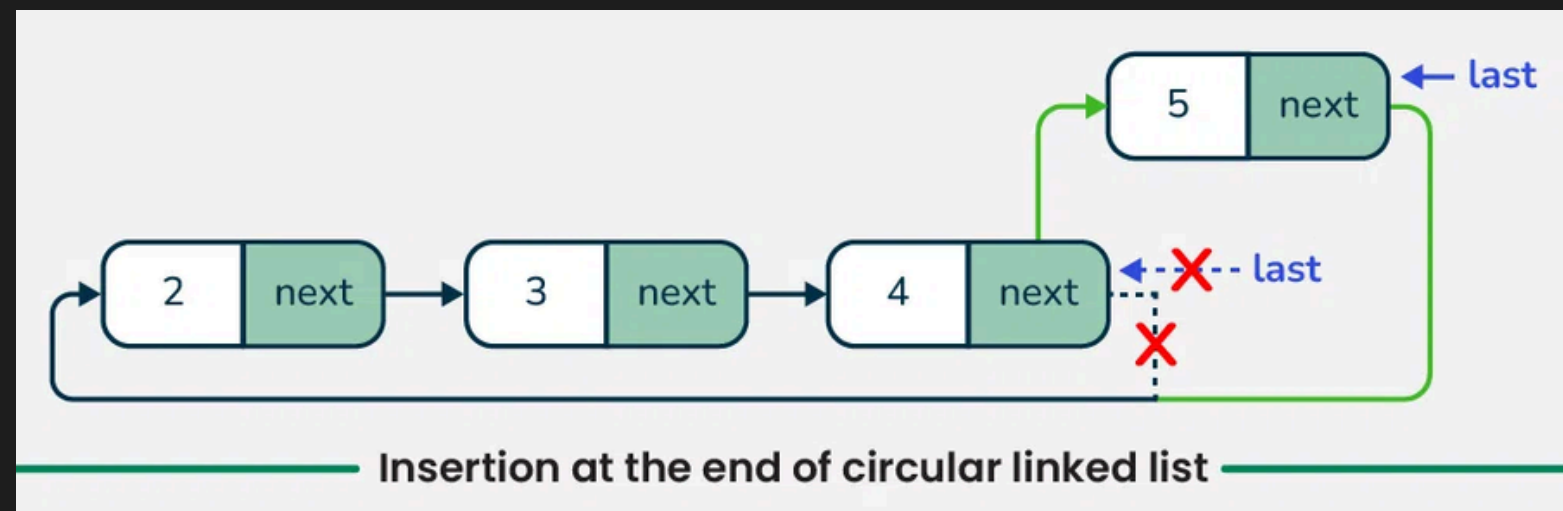
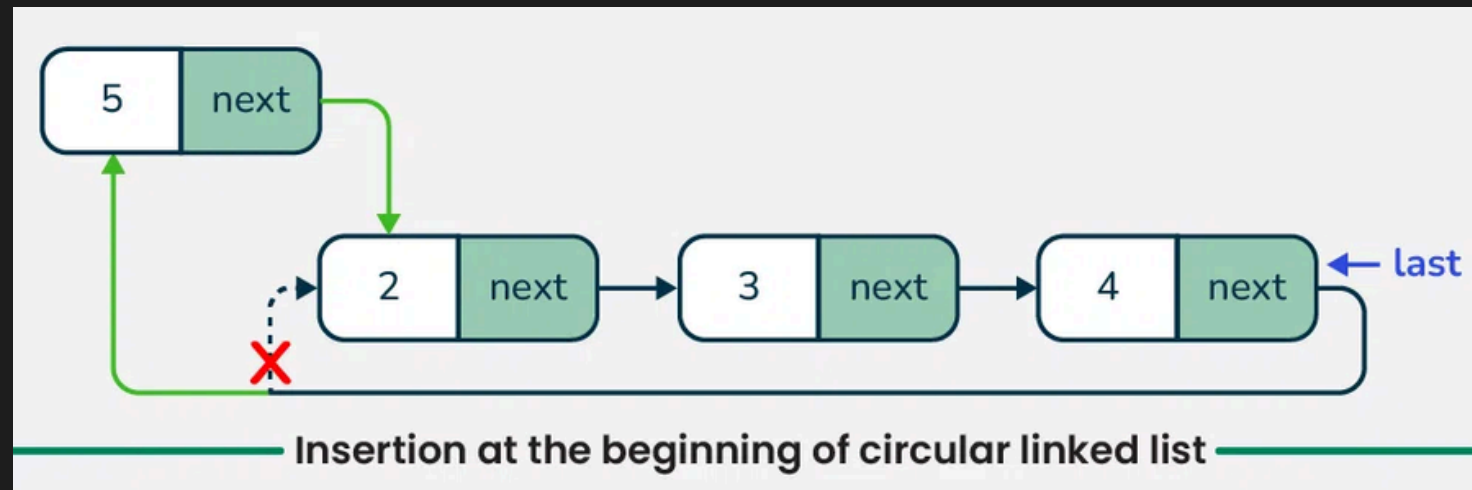
```
struct Student {  
    int id;  
    Student* next;  
};
```



Circular property: $\text{tail} \rightarrow \text{next} == \text{head}$ (non-empty).

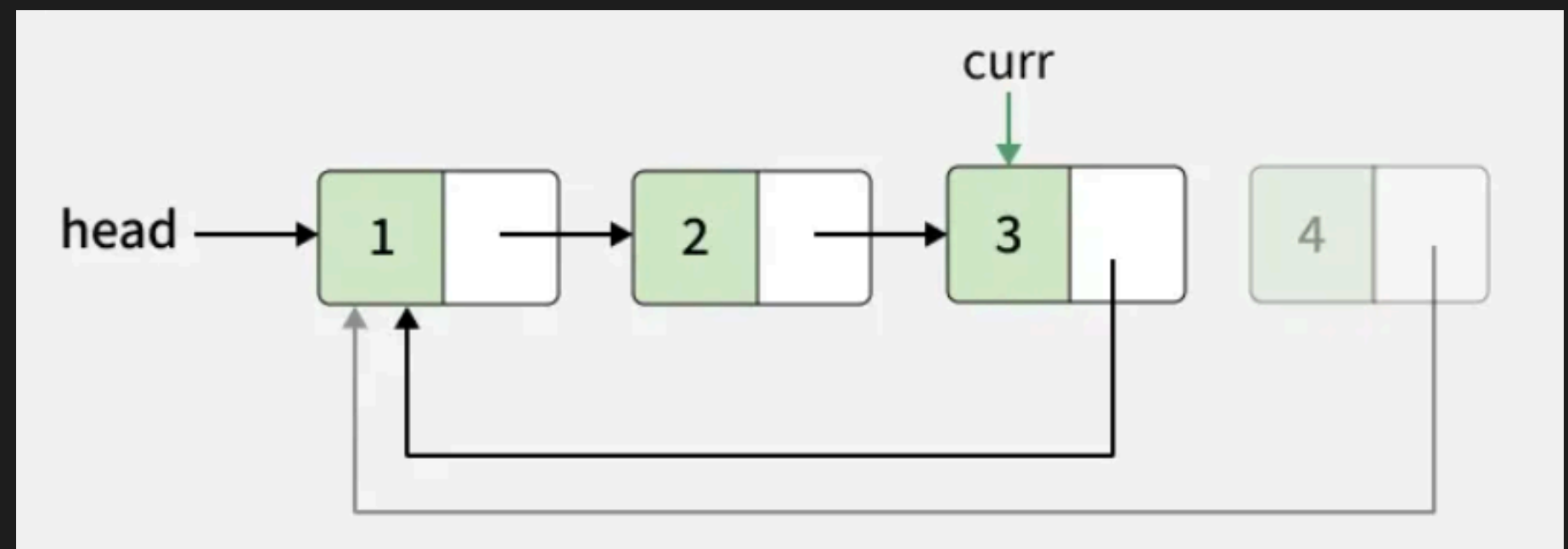
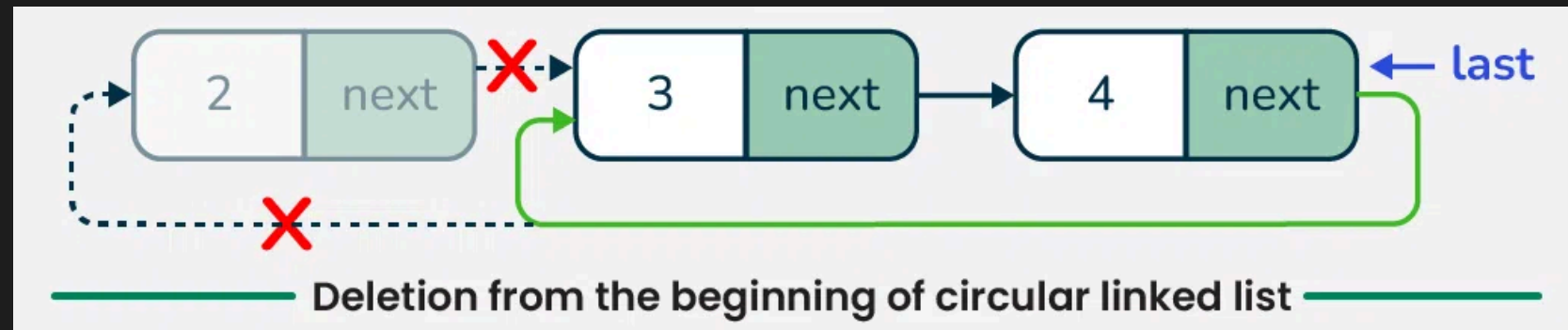


CSLL Insert





CSLL Deletions & Edge Cases





When to Choose CSLL

- Natural cycles (game turns, schedulers, buffers).
- Constant-time wrap-around without index arithmetic.
- Still singly-linked → deleting a known node needs its predecessor (no backward links).

What if we also want $O(1)$ deletion given a direct node pointer, and easy backward traversal?



Link backwards too?

- In CSLL we solved wrap-around; how do we also solve predecessor access?
- If each node kept a pointer to its previous neighbor, which operations become $O(1)$?

Trade-offs:

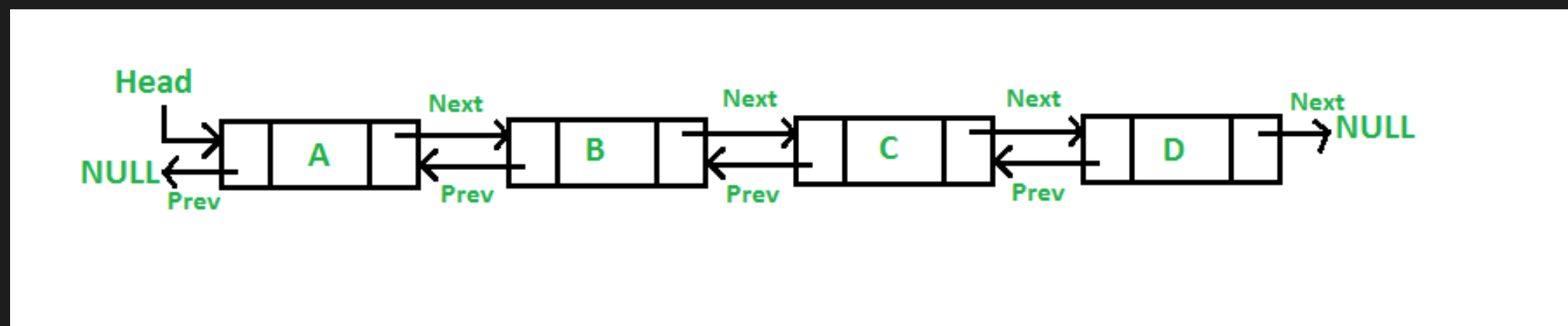
- +backward traversal,
- + $O(1)$ erase by pointer;
- extra pointer/memory,
- more rewiring per op.



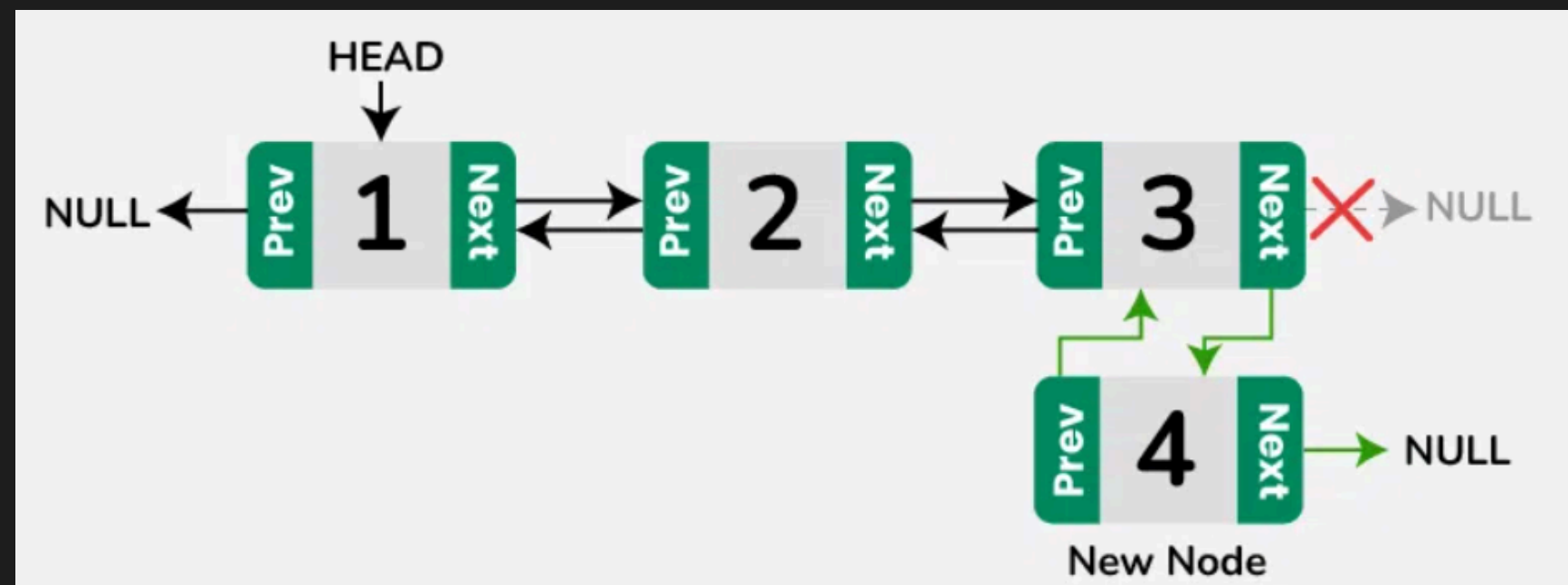
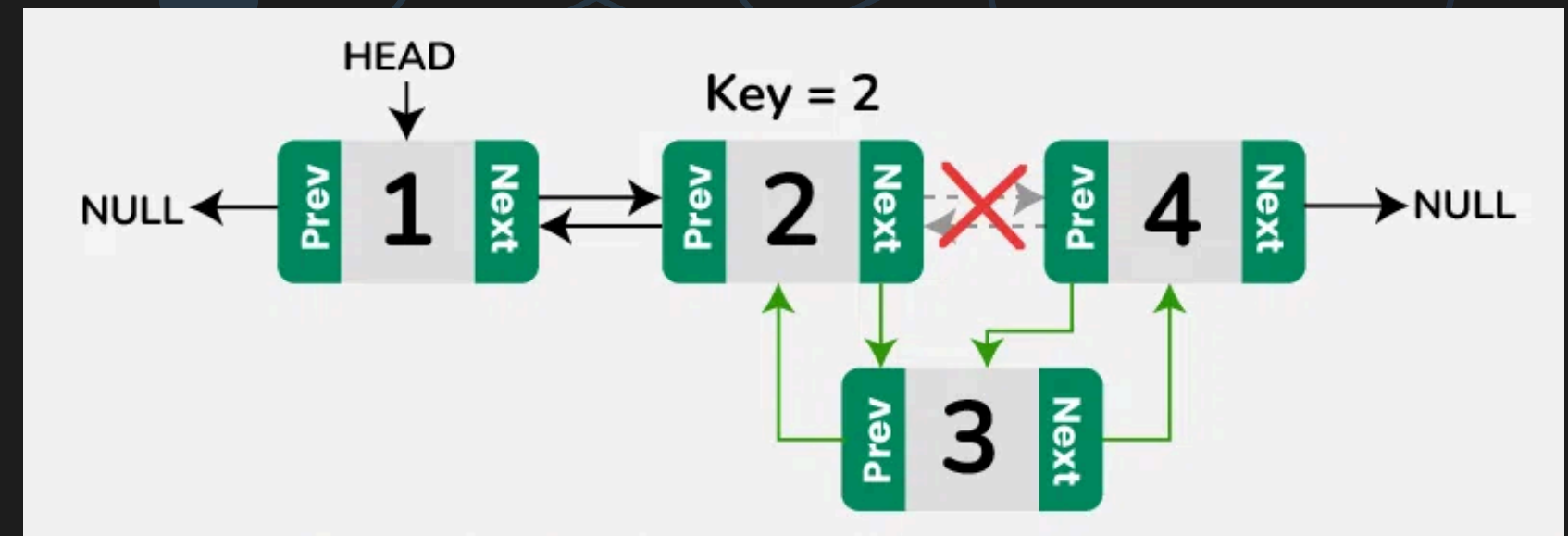
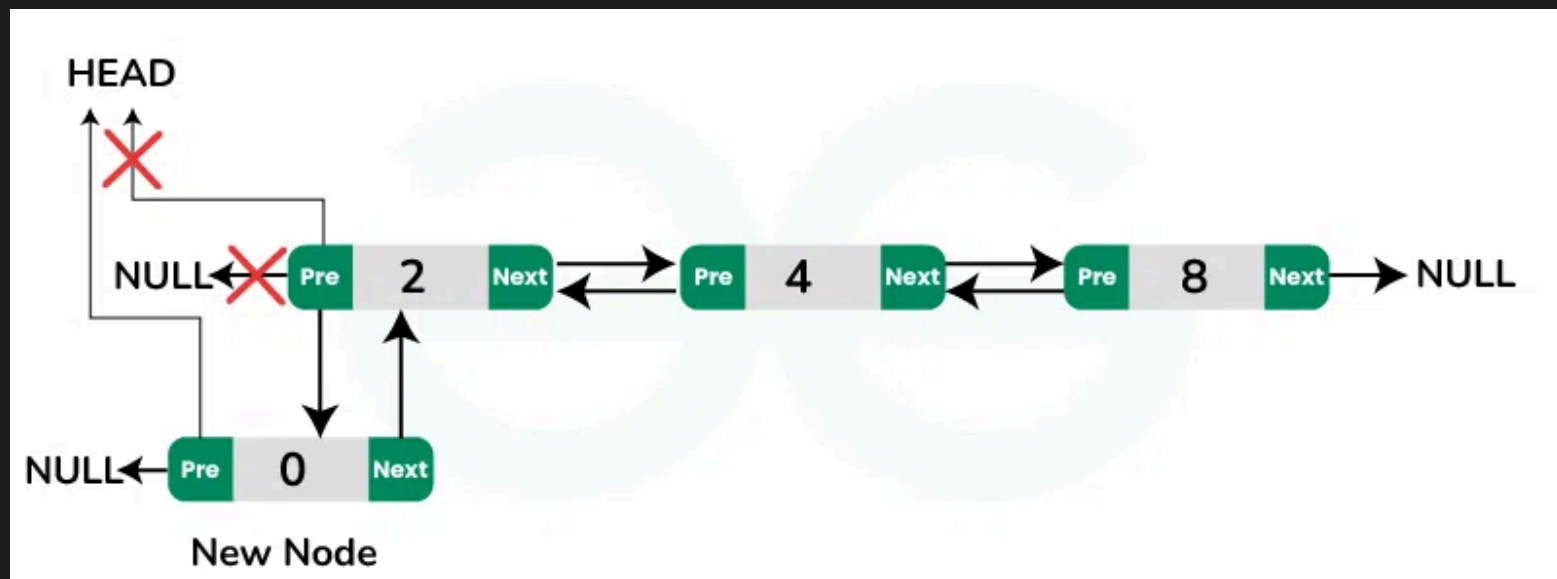
DLL Concept & Node Layout

- Each node knows previous and next.
- Typical list keeps head and often tail pointers.

```
struct Node {  
    int data;  
    Node* prev;  
    Node* next;  
};
```

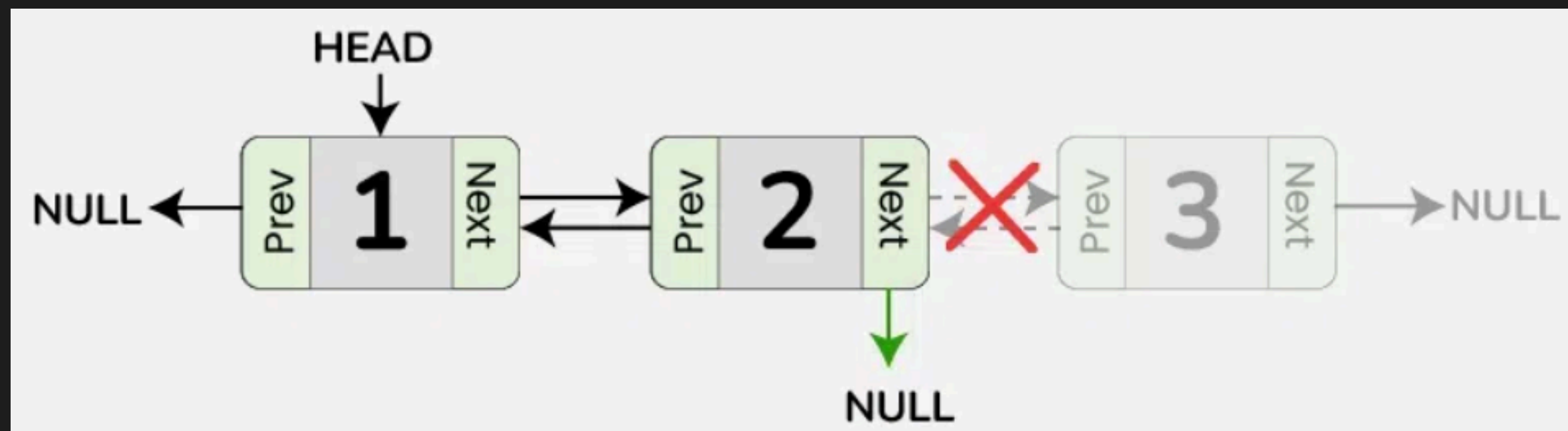
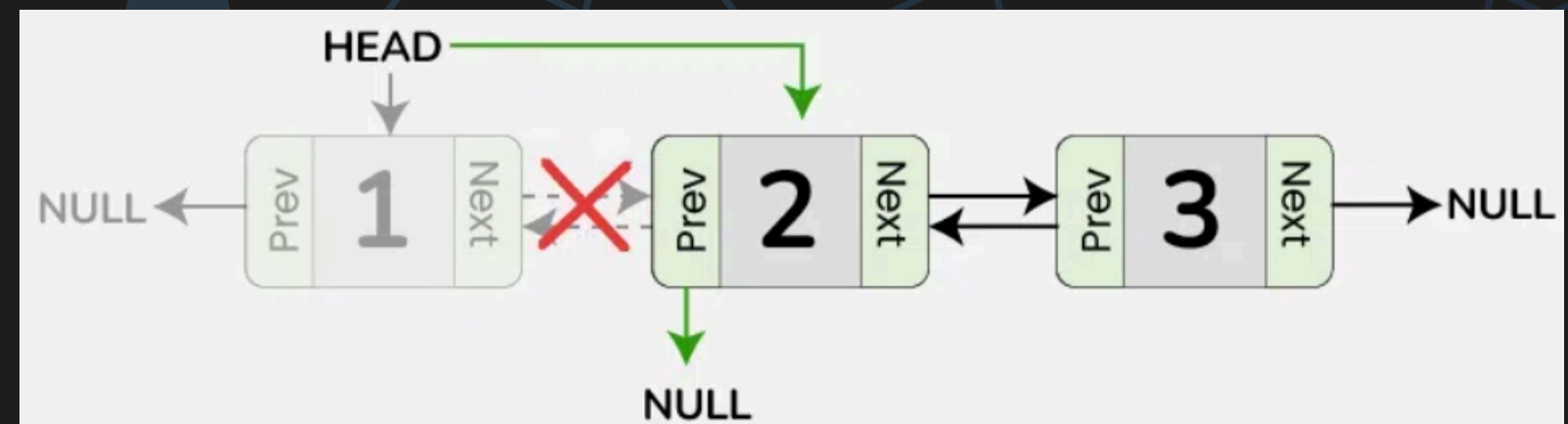
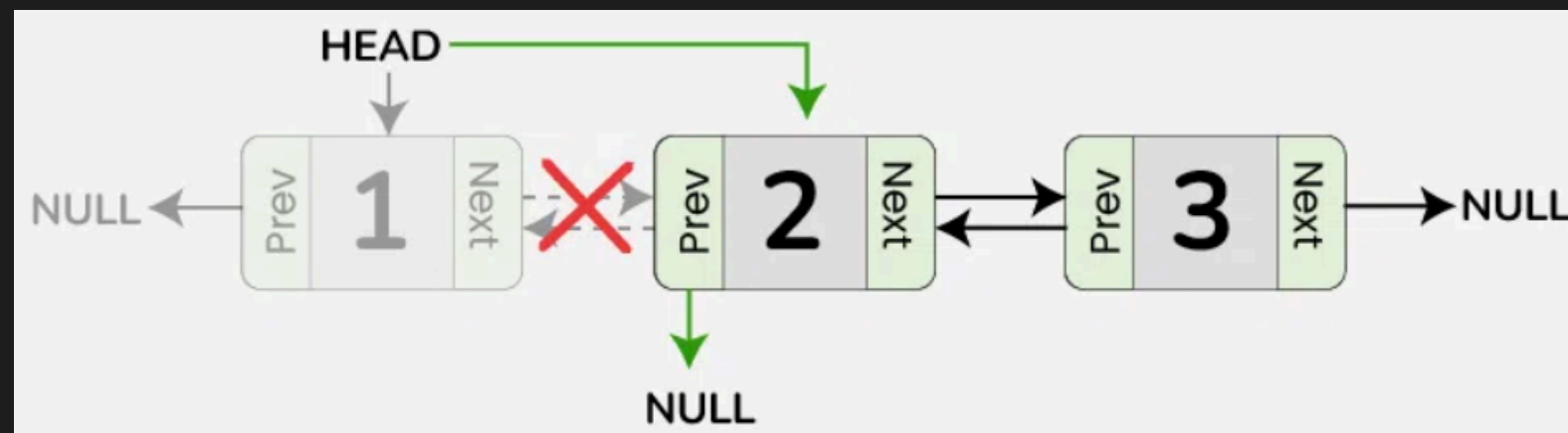


Insertion





Deletion





Insertion & Deletion of Circular DLL

- Insert/delete in the beginning
- Insert/delete in the end
- Insert/delete at specific position



SLL vs CSLL vs DLL vs CDLL

Operation	SLL	CSLL	DLL	CDLL	Array
Push_front					
Push_back					
Insert at Known place					
Random Access					



Conclusion

- **Circular lists**: set tail->next = head to loop seamlessly (great for round-robin/playlists).
- **Singly circular** still can't go backward or delete in $O(1)$ without the predecessor.
- **Doubly linked lists** add prev → easy backward traversal and $O(1)$ delete given a node.
- **Extra cost for DLL**: more memory per node and more pointer updates on insert/delete.
- Keeping both head and tail gives $O(1)$ at both ends (push_front/push_back).
- **Circular + Doubly (CDLL)** = wrap-around + two-way movement; ideal for endless, bidirectional navigation.
-